



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №9
по курсу «Языки и методы программирования»
«Перегрузка операций»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Данная работа предназначена для изучения возможностей языка C++, обеспечивающих применение знаков операций к объектам пользовательских типов.

2 Задание

`Bool<T>` – булевская формула с операциями И, ИЛИ и НЕ, в которой в качестве имён переменных выступают значения типа `T` (это могут быть, например, строки). Операции, перегружаемые для `Bool<T>`:

1. «`>`» и «`+`» – объединение двух формул с помощью операции И и ИЛИ, соответственно; 2. «`!`» – формирование новой формулы, являющейся отрицанием текущей формулы; 3. «`( )`» – вычисление значения формулы по ассоциативному массиву, отображающему имена переменных в булевские значения.

Конструктор класса `Bool<T>` должен принимать имя переменной и формировать формулу, состоящую из обращения к этой единственной переменной. Подразумевается, что более сложные формулы можно собрать с помощью перегруженных операций.

`LazyMatrix<T>` – матрица с элементами типа `T` неопределённого размера, растущая по мере надобности. Требование к типу `T`: наличие конструктора по умолчанию. Операции, которые должны быть перегружены для `LazyMatrix<T>`:

1. «`( )`» – получение ссылки элемент с индексами `i` и `j` (размер матрицы должен быть автоматически увеличен, если `i` или `j` выходят за её пределы); 2. `m` и `n` – возвращают количество строк и столбцов, соответственно; 3. «`!`» – транспонирование матрицы (возвращает новую матрицу). 4. «`==`», «`!=`».

3 Результаты

Исходный код программы представлен в 1, 2, 3, 4.

Листинг 1: Файл

```
1 #include <iostream>
2 #include <map>
3 #include <string>
4 #include "bool.h"
```

```

5
6 int main() {
7     //
8     Bool<std::string> a("a"), b("b");
9     // : XOR = (a AND NOT b) OR (NOT a AND b)
10    auto xor_ab = (a * !b) + (!a * b);
11
12    std::map<std::string, bool> vals;
13    std::cout << "
14    for (bool av : {false, true}) {
15        for (bool bv : {false, true}) {
16            vals["a"] = av;
17            vals["b"] = bv;
18            std::cout << "a=" << std::boolalpha << av
19                << " b=" << bv
20                << " | xor=" << xor_ab(vals)
21                << std::endl;
22        }
23    }
24
25    //
26    Bool<std::string> c("c");
27    // : = NOT(a OR b) AND c
28    auto formula2 = (! (a + b)) * c;
29    //
30
31    vals = {{ "a", false }, { "b", false }, { "c", true }};
32    std::cout << "
33    a=false , b=false , c=true:\n";
34    std::cout << "
35    = " << formula2(vals) << std::endl;
36
37    return 0;
38 }

```

Листинг 2: Файл

```

1 #ifndef BOOL_H
2 #define BOOL_H
3
4 #include <memory>
5 #include <map>
6
7 template<typename T>
8 class Bool {
9     enum Type { VAR, AND, OR, NOT };

```

```

10  struct Node {
11      Type type;
12      T var;
13      std::shared_ptr<Node> left , right ;
14      //
15      Node(const T& v): type(VAR) , var(v) {}
16      //
17      Node(Type t , std::shared_ptr<Node> l , std::shared_ptr<Node> r)
18          : type(t) , left(l) , right(r) {}
19  };
20  std::shared_ptr<Node> root ;
21 public:
22      //
23      Bool(const T& var): root(std::make_shared<Node>(var)) {}
24      //
25      Bool operator*(const Bool& other) const {
26          return Bool(std::make_shared<Node>(AND, root , other.root));
27      }
28      //
29      Bool operator+(const Bool& other) const {
30          return Bool(std::make_shared<Node>(OR, root , other.root));
31      }
32      //
33      Bool operator!() const {
34          return Bool(std::make_shared<Node>(NOT, root , nullptr));
35      }
36      //
37      bool operator()(const std::map<T, bool>& values) const {
38          return eval(root , values);
39      }
40 private:
41      Bool(std::shared_ptr<Node> node): root(node) {}
42      static bool eval(const std::shared_ptr<Node>& node , const std::map<T
43      , bool>& values) {
44          switch (node->type) {
45              case VAR:
46                  if (values.count(node->var)) return values.at(node->var)
47              ;
48                  return false;
49              case AND:
50                  return eval(node->left , values) && eval(node->right ,
51                  values);
52              case OR:
53                  return eval(node->left , values) || eval(node->right ,
54                  values);
55              case NOT:

```

```

52         return !eval(node->left , values);
53     }
54     return false;
55 }
56 };
57
58 #endif // BOOL_H

```

Листинг 3: Файл

```

1  #include <iostream>
2  #include "LazyMatrix.h"
3
4  int main() {
5      LazyMatrix<int> mat;
6      LazyMatrix<int> mat1;
7      LazyMatrix<int> mat2;
8      //
9      mat(0, 1) = 5;
10     mat(2, 0) = 3;
11     mat(3, 4) = 7;
12
13     mat1(0, 0) = 1;
14     mat1(0, 1) = 2;
15     mat1(1, 0) = 3;
16     mat1(1, 1) = 4;
17
18     mat2(0, 0) = 1;
19     mat2(0, 1) = 2;
20     mat2(1, 0) = 0;
21     mat2(1, 1) = 4;
22
23
24     std::cout << "                               : "
25                << mat.m() << "x" << mat.n() << std::endl;
26
27     //
28     std::cout << "                               :\n";
29     for (size_t i = 0; i < mat.m(); ++i) {
30         for (size_t j = 0; j < mat.n(); ++j)
31             std::cout << mat(i,j) << " ";
32         std::cout << "\n";
33     }
34
35     //
36     auto t = !mat;
37     std::cout << "\n

```

```

38     for (size_t i = 0; i < t.m(); ++i) {
39         for (size_t j = 0; j < t.n(); ++j)
40             std::cout << t(i,j) << " ";
41         std::cout << "\n";
42     }
43
44     //
45     std::cout << "\nmat " << (mat == t ? "=" : "!=") << " t\n";
46
47     std::cout << (mat1 == mat2 ? "=" : "!=") << "\n";
48     return 0;
49 }

```

Листинг 4: Файл

```

1  #ifndef LAZYMATRIX_H
2  #define LAZYMATRIX_H
3
4  #include <vector>
5
6  template<typename T>
7  class LazyMatrix {
8      std::vector<std::vector<T>>> data;
9  public :
10      //
11      T& operator () (size_t i, size_t j) {
12          if (i >= data.size()) data.resize(i + 1);
13          if (j >= data[i].size()) data[i].resize(j + 1);
14          return data[i][j];
15      }
16
17      //
18      size_t m() const {
19          return data.size();
20      }
21
22      //
23      size_t n() const {
24          size_t cols = 0;
25          for (const auto& row : data)
26              if (row.size() > cols) cols = row.size();
27          return cols;
28      }
29
30      //

```

```

31     LazyMatrix operator!() const {
32         LazyMatrix<T> res;
33         size_t rows = m(), cols = n();
34         for (size_t i = 0; i < rows; ++i)
35             for (size_t j = 0; j < data[i].size(); ++j)
36                 res(j, i) = data[i][j];
37         return res;
38     }
39
40     //
41     bool operator==(const LazyMatrix& other) const {
42         if (m() != other.m() || n() != other.n()) return false;
43         for (size_t i = 0; i < m(); ++i)
44             for (size_t j = 0; j < other.data[i].size(); ++j)
45                 if (data[i][j] != other.data[i][j])
46                     return false;
47         return true;
48     }
49
50     //
51     bool operator!=(const LazyMatrix& other) const {
52         return !(*this == other);
53     }
54 };
55
56 #endif // LAZYMATRIX_H

```

Результат запуска представлен на рисунке 1.

```

Ср 25 июн - 21:55 ~
@keiichi> cd cpp_yannaev

Ср 25 июн - 21:55 ~/cpp_yannaev
@keiichi> cd lab_9

Ср 25 июн - 21:55 ~/cpp_yannaev/lab_9
@keiichi> ls
var_1  var_24

Ср 25 июн - 21:55 ~/cpp_yannaev/lab_9
@keiichi> cd var_1

Ср 25 июн - 21:55 ~/cpp_yannaev/lab_9/var_1
@keiichi> ./bool
таблица истинности для XOR (a, b):
a=false b=false | xor=false
a=false b=true  | xor=true
a=true  b=false | xor=true
a=true  b=true  | xor=false

пример 3-переменной формулы при a=false, b=false, c=true:
результат = true

Ср 25 июн - 21:56 ~/cpp_yannaev/lab_9/var_1
@keiichi> cd ..

Ср 25 июн - 21:56 ~/cpp_yannaev/lab_9
@keiichi> cd var_24

Ср 25 июн - 21:56 ~/cpp_yannaev/lab_9/var_24
@keiichi> ./LazyMatrix
размеры матрицы: 4x5
оригинал:
0 5 0 0 0
0 0 0 0 0
3 0 0 0 0
0 0 0 0 7

транспонированная:
0 0 3 0
5 0 0 0
0 0 0 0
0 0 0 0
0 0 0 7

mat ≠ t
≠%

Ср 25 июн - 21:56 ~/cpp_yannaev/lab_9/var_24
@keiichi> |

```

Рис. 1 — Результат